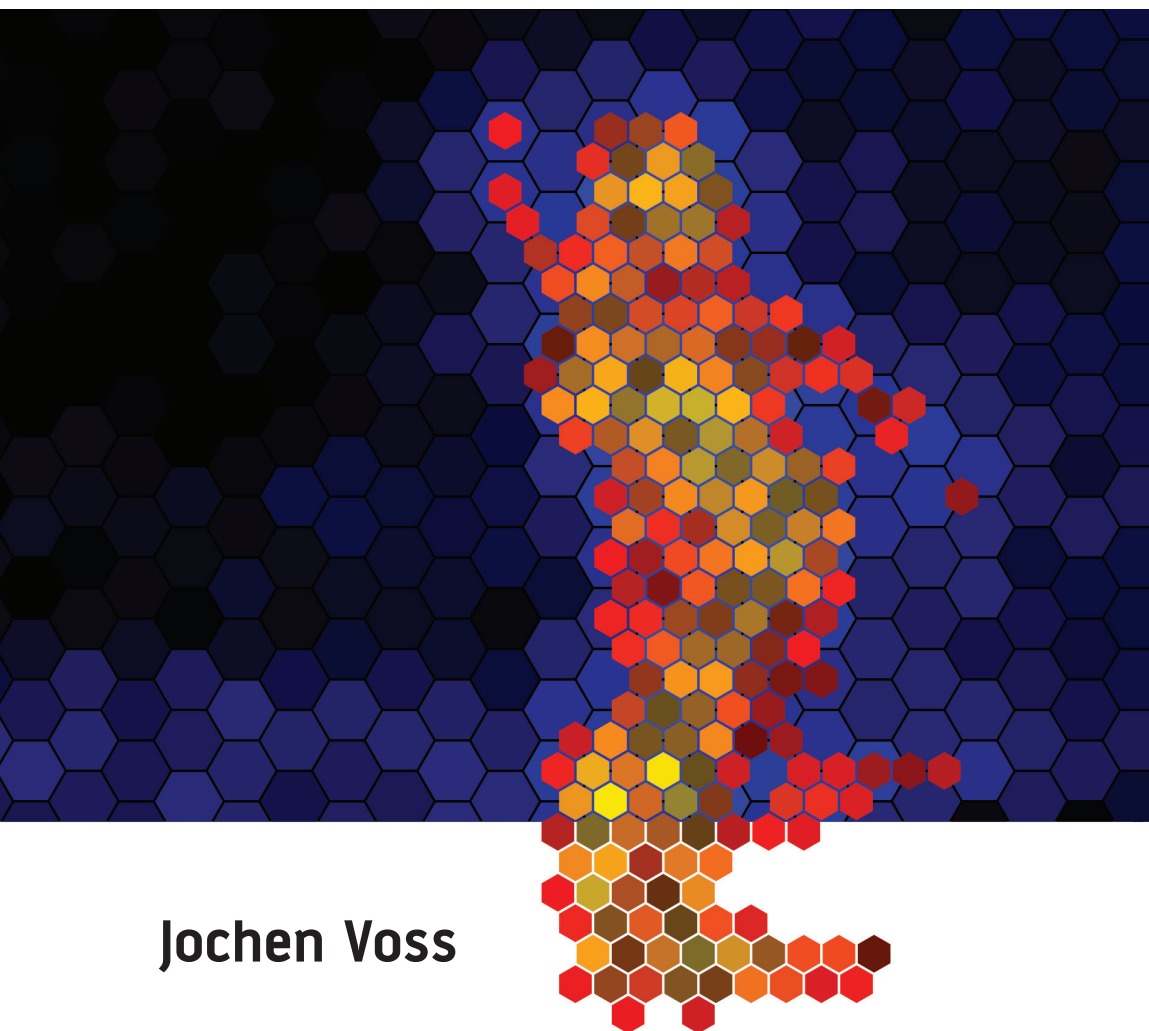


WILEY SERIES IN COMPUTATIONAL STATISTICS



Jochen Voss

AN INTRODUCTION TO
**STATISTICAL
COMPUTING**

A SIMULATION-BASED APPROACH

WILEY

An Introduction to Statistical Computing

WILEY SERIES IN COMPUTATIONAL STATISTICS

Consulting Editors:

Paolo Giudici
University of Pavia, Italy

Geof H. Givens
Colorado State University, USA

Bani K. Mallick
Texas A & M University, USA

Wiley Series in Computational Statistics is comprised of practical guides and cutting edge research books on new developments in computational statistics. It features quality authors with a strong applications focus. The texts in the series provide detailed coverage of statistical concepts, methods and case studies in areas at the interface of statistics, computing, and numerics.

With sound motivation and a wealth of practical examples, the books show in concrete terms how to select and to use appropriate ranges of statistical computing techniques in particular fields of study. Readers are assumed to have a basic understanding of introductory terminology.

The series concentrates on applications of computational methods in statistics to fields of bioinformatics, genomics, epidemiology, business, engineering, finance and applied statistics.

Titles in the Series

Biegler, Biros, Ghattas, Heinkenschloss, Keyes, Mallick, Marzouk, Tenorio, Waanders, Willcox – Large-Scale Inverse Problems and Quantification of Uncertainty

Billard and Diday – Symbolic Data Analysis: Conceptual Statistics and Data Mining

Bolstad – Understanding Computational Bayesian Statistics

Borgelt, Steinbrecher and Kruse – Graphical Models, 2e

Dunne – A Statistical Approach to Neural Networks for Pattern Recognition

Liang, Liu and Carroll – Advanced Markov Chain Monte Carlo Methods

Ntzoufras – Bayesian Modeling Using WinBUGS

Tufféry – Data Mining and Statistics for Decision Making

An Introduction to Statistical Computing

A Simulation-based Approach

Jochen Voss

School of Mathematics, University of Leeds, UK

WILEY

This edition first published 2014
© 2014 John Wiley & Sons, Ltd

Registered office

John Wiley & Sons, Ltd, The Atrium, Southern Gate, Chichester, West Sussex, PO19 8SQ,
United Kingdom

For details of our global editorial offices, for customer services and for information about how to apply for permission to reuse the copyright material in this book please see our website at www.wiley.com.

The right of the author to be identified as the author of this work has been asserted in accordance with the Copyright, Designs and Patents Act 1988.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, except as permitted by the UK Copyright, Designs and Patents Act 1988, without the prior permission of the publisher.

Wiley also publishes its books in a variety of electronic formats. Some content that appears in print may not be available in electronic books.

Designations used by companies to distinguish their products are often claimed as trademarks. All brand names and product names used in this book are trade names, service marks, trademarks or registered trademarks of their respective owners. The publisher is not associated with any product or vendor mentioned in this book.

Limit of Liability/Disclaimer of Warranty: While the publisher and author have used their best efforts in preparing this book, they make no representations or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaim any implied warranties of merchantability or fitness for a particular purpose. It is sold on the understanding that the publisher is not engaged in rendering professional services and neither the publisher nor the author shall be liable for damages arising herefrom. If professional advice or other expert assistance is required, the services of a competent professional should be sought.

Library of Congress Cataloging-in-Publication Data

Voss, Jochen.

An introduction to statistical computing : a simulation-based approach / Jochen Voss. – First edition.
pages cm. – (Wiley series in computational statistics)

Includes bibliographical references and index.

ISBN 978-1-118-35772-9 (hardback)

1. Mathematical statistics—Data processing. I. Title.

QA276.4.V66 2013

519.501'13—dc23

2013019321

A catalogue record for this book is available from the British Library.

ISBN: 978-1-118-35772-9

Typeset in 10/12pt Times by Aptara Inc., New Delhi, India

Contents

List of algorithms	ix
Preface	xi
Nomenclature	xiii
1 Random number generation	1
1.1 Pseudo random number generators	2
1.1.1 The linear congruential generator	2
1.1.2 Quality of pseudo random number generators	4
1.1.3 Pseudo random number generators in practice	8
1.2 Discrete distributions	8
1.3 The inverse transform method	11
1.4 Rejection sampling	15
1.4.1 Basic rejection sampling	15
1.4.2 Envelope rejection sampling	18
1.4.3 Conditional distributions	22
1.4.4 Geometric interpretation	26
1.5 Transformation of random variables	30
1.6 Special-purpose methods	36
1.7 Summary and further reading	36
Exercises	37
2 Simulating statistical models	41
2.1 Multivariate normal distributions	41
2.2 Hierarchical models	45
2.3 Markov chains	50
2.3.1 Discrete state space	51
2.3.2 Continuous state space	56
2.4 Poisson processes	58
2.5 Summary and further reading	67
Exercises	67

3	Monte Carlo methods	69
3.1	Studying models via simulation	69
3.2	Monte Carlo estimates	74
3.2.1	Computing Monte Carlo estimates	75
3.2.2	Monte Carlo error	76
3.2.3	Choice of sample size	80
3.2.4	Refined error bounds	82
3.3	Variance reduction methods	84
3.3.1	Importance sampling	84
3.3.2	Antithetic variables	88
3.3.3	Control variates	93
3.4	Applications to statistical inference	96
3.4.1	Point estimators	97
3.4.2	Confidence intervals	100
3.4.3	Hypothesis tests	103
3.5	Summary and further reading	106
	Exercises	106
4	Markov Chain Monte Carlo methods	109
4.1	The Metropolis–Hastings method	110
4.1.1	Continuous state space	110
4.1.2	Discrete state space	113
4.1.3	Random walk Metropolis sampling	116
4.1.4	The independence sampler	119
4.1.5	Metropolis–Hastings with different move types	120
4.2	Convergence of Markov Chain Monte Carlo methods	125
4.2.1	Theoretical results	125
4.2.2	Practical considerations	129
4.3	Applications to Bayesian inference	137
4.4	The Gibbs sampler	141
4.4.1	Description of the method	141
4.4.2	Application to parameter estimation	146
4.4.3	Applications to image processing	151
4.5	Reversible Jump Markov Chain Monte Carlo	158
4.5.1	Description of the method	160
4.5.2	Bayesian inference for mixture distributions	171
4.6	Summary and further reading	178
4.6	Exercises	178
5	Beyond Monte Carlo	181
5.1	Approximate Bayesian Computation	181
5.1.1	Basic Approximate Bayesian Computation	182
5.1.2	Approximate Bayesian Computation with regression	188
5.2	Resampling methods	192

5.2.1	Bootstrap estimates	192
5.2.2	Applications to statistical inference	197
5.3	Summary and further reading	209
	Exercises	209
6	Continuous-time models	213
6.1	Time discretisation	213
6.2	Brownian motion	214
6.2.1	Properties	216
6.2.2	Direct simulation	217
6.2.3	Interpolation and Brownian bridges	218
6.3	Geometric Brownian motion	221
6.4	Stochastic differential equations	224
6.4.1	Introduction	224
6.4.2	Stochastic analysis	226
6.4.3	Discretisation schemes	231
6.4.4	Discretisation error	236
6.5	Monte Carlo estimates	243
6.5.1	Basic Monte Carlo	243
6.5.2	Variance reduction methods	247
6.5.3	Multilevel Monte Carlo estimates	250
6.6	Application to option pricing	255
6.7	Summary and further reading	259
	Exercises	260
Appendix A	Probability reminders	263
A.1	Events and probability	263
A.2	Conditional probability	266
A.3	Expectation	268
A.4	Limit theorems	269
A.5	Further reading	270
Appendix B	Programming in R	271
B.1	General advice	271
B.2	R as a Calculator	272
B.2.1	Mathematical operations	273
B.2.2	Variables	273
B.2.3	Data types	275
B.3	Programming principles	282
B.3.1	Don't repeat yourself!	283
B.3.2	Divide and conquer!	286
B.3.3	Test your code!	290
B.4	Random number generation	292
B.5	Summary and further reading	294
	Exercises	294

Appendix C	Answers to the exercises	299
C.1	Answers for Chapter 1	299
C.2	Answers for Chapter 2	315
C.3	Answers for Chapter 3	319
C.4	Answers for Chapter 4	328
C.5	Answers for Chapter 5	342
C.6	Answers for Chapter 6	350
C.7	Answers for Appendix B	366
References		375
Index		379

List of algorithms

Random number generation

alg. 1.2	linear congruential generator	2
alg. 1.13	inverse transform method	12
alg. 1.19	basic rejection sampling	15
alg. 1.22	envelope rejection sampling	19
alg. 1.25	rejection sampling for conditional distributions	22

Simulating statistical models

alg. 2.9	mixture distributions	47
alg. 2.11	componentwise simulation	49
alg. 2.22	Markov chains with discrete state space	53
alg. 2.31	Markov chains with continuous state space	58
alg. 2.36	Poisson process	61
alg. 2.41	thinning method for Poisson processes	65

Monte Carlo methods

alg. 3.8	Monte Carlo estimate	75
alg. 3.22	importance sampling	85
alg. 3.26	antithetic variables	89
alg. 3.31	control variates	93

Markov Chain Monte Carlo methods

alg. 4.2	Metropolis–Hastings method for continuous state space	110
alg. 4.4	Metropolis–Hastings method for discrete state space	113
alg. 4.9	random walk Metropolis	117
alg. 4.11	independence sampler	119
alg. 4.12	Metropolis–Hastings method with different move types	121
alg. 4.27	Gibbs sampler	142
alg. 4.31	Gibbs sampler for the Ising model	155
alg. 4.32	Gibbs sampler in image processing	158
alg. 4.36	reversible jump Markov Chain Monte Carlo	165

Beyond Monte Carlo

alg. 5.1	basic Approximate Bayesian Computation	182
alg. 5.6	Approximate Bayesian Computation with regression	191
alg. 5.11	general bootstrap estimate	196
alg. 5.15	bootstrap estimate of the bias	200
alg. 5.18	bootstrap estimate of the standard error	202
alg. 5.20	simple bootstrap confidence interval	205
alg. 5.21	BC _a bootstrap confidence interval	207

Continuous-time models

alg. 6.6	Brownian motion	217
alg. 6.12	Euler–Maruyama scheme	232
alg. 6.15	Milstein scheme	235
alg. 6.26	multilevel Monte Carlo estimates	251
alg. 6.29	Euler–Maruyama scheme for the Heston model	256

Preface

This is a book about exploring random systems using computer simulation and thus, this book combines two different topic areas which have always fascinated me: the mathematical theory of probability and the art of programming computers. The method of using computer simulations to study a system is very different from the more traditional, purely mathematical approach. On the one hand, computer experiments normally can only provide approximate answers to quantitative questions, but on the other hand, results can be obtained for a much wider class of systems, including large and complex systems where a purely theoretical approach becomes difficult.

In this text we will focus on three different types of questions. The first, easiest question is about the normal behaviour of the system: what is a typical state of the system? Such questions can be easily answered using computer experiments: simulating a few random samples of the system gives examples of typical behaviour. The second kind of question is about variability: how large are the random fluctuations? This type of question can be answered statistically by analysing large samples, generated using repeated computer simulations. A final, more complicated class of questions is about exceptional behaviour: how small is the probability of the system behaving in a specified untypical way? Often, advanced methods are required to answer this third type of question. The purpose of this book is to explain how such questions can be answered. My hope is that, after reading this book, the reader will not only be able to confidently use methods from statistical computing for answering such questions, but also to adjust existing methods to the requirements of a given problem and, for use in more complex situations, to develop new specialised variants of the existing methods.

This text originated as a set of handwritten notes which I used for teaching the ‘Statistical Computing’ module at the University of Leeds, but now is greatly extended by the addition of many examples and more advanced topics. The material we managed to cover in the ‘Statistical Computing’ course during one semester is less than half of what is now the contents of the book! This book is aimed at postgraduate students and their lecturers; it can be used both for self-study and as the basis of taught courses. With the inclusion of many examples and exercises, the text should also be accessible to interested undergraduate students and to mathematically inclined researchers from areas outside mathematics.

Only very few prerequisites are required for this book. On the mathematical side, the text assumes that the reader is familiar with basic probability, up to and including the law of large numbers; Appendix A summarises the required results. As a consequence of the decision to require so little mathematical background, some of the finer mathematical subtleties are not discussed in this book. Results are presented in a way which makes them easily accessible to readers with limited mathematical background, but the statements are given in a form which allows the mathematically more knowledgeable reader to easily add the required detail on his/her own. (For example, I often use phrases such as ‘every set $A \subseteq \mathbb{R}^d$ ’, where full mathematical rigour would require us to write ‘every *measurable* set $A \subseteq \mathbb{R}^d$ ’.) On the computational side, basic programming skills are required to make use of the numerical methods introduced in this book. While the text is written independent of any specific programming language, the reader will need to choose a language when implementing methods from this book on a computer. Possible choices of programming language include Python, Matlab and C/C++. For my own implementations, provided as part of the solutions to the exercises in Appendix C, I used the R programming language; a short introduction to programming with R is provided in Appendix B.

Writing this book has been a big adventure for me. When I started this project, more than a year ago, my aim was to cover enough material so that I could discuss the topics of multilevel Monte Carlo and reversible jump Markov Chain Monte Carlo methods. I estimated that 350 pages would be enough to cover this material but it quickly transpired that I had been much too optimistic: my estimates for the final page count kept rising and even after several rounds of throwing out side-topics and generally tightening the text, the book is still stretching this limit! Nevertheless, the text now covers most of the originally planned topics, including multilevel Monte Carlo methods near the very end of the book. Due to my travel during the last year, parts of this book have been written on a laptop in exciting places. For example, the initial draft of section 1.5 was written on a coach travelling through the beautiful island of Kyushu, halfway around the world from where I live! All in all, I greatly enjoyed writing this book and I hope that the result is useful to the reader.

This book contains an accompanying website. Please visit www.wiley.com/go/statistical_computing

Jochen Voss
Leeds, March 2013

Nomenclature

For reference, the following list summarises some of the notation used throughout this book.

$\emptyset$	the empty set
$\mathbb{N}$	the natural numbers: $\mathbb{N} = \{1, 2, 3, \dots\}$
$\mathbb{N}_0$	the non-negative integers: $\mathbb{N} = \{0, 1, 2, \dots\}$
$\mathbb{Z}$	the integers: $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
$n \bmod m$	the remainder of the division of n by m , in the range $0, 1, \dots, m - 1$
δ_{kl}	the Kronecker delta: $\delta_{kl} = 1$ if $k = l$ and $\delta_{kl} = 0$ otherwise
$\mathbb{R}$	the real numbers
$\lceil x \rceil$	the number $x \in \mathbb{R}$ ‘rounded up’, that is the smallest integer greater than or equal to x
$(a_n)_{n \in \mathbb{N}}$	a sequence of (possibly random) numbers: $(a_n)_{n \in \mathbb{N}} = (a_1, a_2, \dots)$
$\mathcal{O}(\cdot)$	the big $\mathcal{O}$ notation, introduced in definition 3.16
$[a, b]$	an interval of real numbers: $[a, b] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$
$\{a, b\}$	the set containing a and b
A^c	the complement of a set: $A^c = \{x \mid x \notin A\}$.
$A \times B$	the Cartesian product of the sets A and B : $A \times B = \{(a, b) \mid a \in A, b \in B\}$
$\mathbb{1}_A(x)$	the indicator function of the set A : $\mathbb{1}_A(x) = 1$ if $x \in A$ and 0 otherwise (see section A.3)
$\mathcal{U}[0, 1]$	the uniform distribution on the interval $[0, 1]$
$\mathcal{U}\{-1, 1\}$	the uniform distribution on the two-element set $\{-1, 1\}$
$\text{Pois}(\lambda)$	the Poisson distribution with parameter λ
$X \sim \mu$	indicates that a random variable X is distributed according to a probability distribution μ
$ S $	the number of elements in a finite set S ; in section 1.4 also the volume of a subsets $S \subseteq \mathbb{R}^d$
$\mathbb{R}^S$	space of vectors where the components are indexed by elements of S (see section 2.3.2)
$\mathbb{R}^{S \times S}$	space of matrices where rows and columns are indexed by elements of S (see section 2.3.2)

1

Random number generation

The topic of this book is the study of statistical models using computer simulations. Here we use the term ‘statistical models’ to mean any mathematical models which include a random component. Our interest in this chapter and the next is in simulation of the random component of these models. The basic building block of such simulations is the ability to generate random numbers on a computer, and this is the topic of the present chapter. Later, in Chapter 2, we will see how the methods from Chapter 1 can be combined to simulate more complicated models.

Generation of random numbers, or more general random objects, on a computer is complicated by the fact that computer programs are inherently deterministic: while the output of computer program may look random, it is obtained by executing the steps of some algorithm and thus is totally predictable. For example the output of a program computing the decimal digits of the number

$$\pi = 3.14159265358979323846264338327950288419716939937510 \dots$$

(the ratio between the perimeter and diameter of a circle) looks random at first sight, but of course π is not random at all! The output can only start with the string of digits given above and running the program twice will give the same output twice.

We will split the problem of generating random numbers into two distinct sub-problems: first we will study the problem of generating any randomness at all, concentrating on the simple case of generating independent random numbers, uniformly distributed on the interval $[0, 1]$. This problem and related concerns will be discussed in Section 1.1. In the following sections, starting with Section 1.2, we will study the generation of random numbers from different distributions, using the independent, uniformly distributed random numbers obtained in the previous step as a basis.

1.1 Pseudo random number generators

There are two fundamentally different classes of methods to generate random numbers:

- (a) True random numbers are generated using some physical phenomenon which is random. Generating such numbers requires specialised hardware and can be expensive and slow. Classical examples of this include tossing a coin or throwing dice. Modern methods utilise quantum effects, thermal noise in electric circuits, the timing of radioactive decay, etc.
- (b) Pseudo random numbers are generated by computer programs. While these methods are normally fast and resource effective, a challenge with this approach is that computer programs are inherently deterministic and therefore cannot produce ‘truly random’ output.

In this text we will only consider pseudo random number generators.

Definition 1.1 A pseudo random number generator (PRNG) is an algorithm which outputs a sequence of numbers that can be used as a replacement for an independent and identically distributed (i.i.d.) sequence of ‘true random numbers’.

1.1.1 The linear congruential generator

This section introduces the linear congruential generator (LCG), a simple example of a PRNG. While this random number generator is no longer of practical importance, it shares important characteristics with the more complicated generators used in practice today and we study it here as an accessible example. The LCG is given by the following algorithm.

Algorithm 1.2 (linear congruential generator)

input:

$m > 1$ (the *modulus*)

$a \in \{1, 2, \dots, m-1\}$ (the *multiplier*)

$c \in \{0, 1, \dots, m-1\}$ (the *increment*)

$X_0 \in \{0, 1, \dots, m-1\}$ (the *seed*)

output:

a sequence $X_1, X_2, X_3, \dots$ of pseud random numbers

1: **for** $n = 1, 2, 3, \dots$ **do**

2: $X_n \leftarrow (aX_{n-1} + c) \bmod m$

3: output X_n

4: **end for**

In the algorithm, ‘mod’ denotes the modulus for integer division, that is the value $n \bmod m$ is the remainder of the division of n by m , in the range $0, 1, \dots, m - 1$. Thus the sequence generated by algorithm 1.2 consists of integers X_n from the range $\{0, 1, 2, \dots, m - 1\}$. The output depends on the parameters m, a, c and on the seed X_0 . We will see that, if m, a and c are carefully chosen, the resulting sequence behaves ‘similar’ to a sequence of independent, uniformly distributed random variables. By choosing different values for the seed X_0 , different sequences of pseudo random numbers can be obtained.

Example 1.3 For parameters $m = 8, a = 5, c = 1$ and seed $X_0 = 0$, algorithm 1.2 gives the following output:

n	$5X_{n-1} + 1$	X_n
1	1	1
2	6	6
3	31	7
4	36	4
5	21	5
6	26	2
7	11	3
8	16	0
9	1	1
10	6	6

The output 1, 6, 7, 4, 5, 2, 3, 0, 1, 6, ... shows no obvious pattern and could be considered to be a sample of a random sequence.

While the output of the LCG looks random, from the way it is generated it is clear that the output has several properties which make it different from truly random sequences. For example, since each new value of X_n is computed from X_{n-1} , once the generated series reaches a value X_n which has been generated before, the output starts to repeat. In example 1.3 this happens for $X_8 = X_0$ and we get $X_9 = X_1, X_{10} = X_2$ and so on. Since X_n can take only m different values, the output of a LCG starts repeating itself after at most m steps; the generated sequence is eventually periodic.

Sometimes the periodicity of a sequence of pseudo random numbers can cause problems, but on the other hand, if the period length is longer than the amount of random numbers we use, periodicity cannot affect our result. For this reason, one needs to carefully choose the parameters m, a and c in order to achieve a long enough period. In particular m , since it is an upper bound for the period length, needs to be chosen large. In practice, typical values of m are on the order of $m = 2^{32} \approx 4 \cdot 10^9$ and a and c are then chosen such that the generator actually achieves the maximally possible period length of m . A criterion for the choice of m, a and c is given in the following theorem (Knuth, 1981, Section 3.2.1.2).

Theorem 1.4 The LCG has period m if and only if the following three conditions are satisfied:

- (a) m and c are relatively prime;
- (b) $a - 1$ is divisible by every prime factor of m ;
- (c) if m is a multiple of 4, then $a - 1$ is a multiple of 4.

In the situation of the theorem, the period length does not depend on the seed X_0 and usually this parameter is left to be chosen by the user of the PRNG.

Example 1.5 Let $m = 2^{32}$, $a = 1\,103\,515\,245$ and $c = 12\,345$. Since the only prime factor of m is 2 and c is odd, the values m and c are relatively prime and condition (a) of the theorem is satisfied. Similarly, condition (b) is satisfied, since $a - 1$ is even and thus divisible by 2. Finally, since m is a multiple of 4, we have to check condition (c) but, since $a - 1 = 1\,103\,515\,244 = 275\,878\,811 \cdot 4$, this condition also holds. Therefore the LCG with these parameters m , a and c has period 2^{32} for every seed X_0 .

1.1.2 Quality of pseudo random number generators

PRNGs used in modern software packages such as R or Matlab are more sophisticated (and more complicated) than the LCG presented in Section 1.1.1, but they still share many characteristics of the LCG. We will see that no PRNG can produce a perfect result, but the random number generators used in practice, for example the Mersenne Twister algorithm (Matsumoto and Nishimura, 1998), are good enough for most purposes. In this section we will discuss criteria for the quality of the output of general PRNGs, and will illustrate these criteria using the LCG as an example.

1.1.2.1 Period length of the output

We have seen that the output of the LCG is eventually periodic, with a period length of at most m . This property that the output is eventually periodic is shared by all PRNGs implemented in software. Most PRNGs used in practice have a period length which is much larger than the amount of random numbers a computer program could ever use in a reasonable time. For this reason, periodicity of the output is not a big problem in practical applications of PRNGs. The period length is a measure for the quality of a PRNG.

1.1.2.2 Distribution of samples

The output of almost all PRNGs is constructed so that it can be used as a replacement for an i.i.d. sample of *uniformly distributed* random numbers. Since the output takes

values in a finite set $S = \{0, 1, \dots, m-1\}$, in the long run, for every set $A \subseteq S$ we should have

$$\frac{\#\{i \mid 1 \leq i \leq N, X_i \in A\}}{N} \approx \frac{\#A}{\#S}, \quad (1.1)$$

where $\#A$ stands for the number of elements in a finite set A .

Uniformity of the output can be tested using statistical tests like the chi-squared test or the Kolmogorov–Smirnov test (see e.g. Lehmann and Romano, 2005, Chapter 14).

One peculiarity when applying statistical tests for the distribution of samples to the output of a PRNG is that the test may fail in two different ways: The output could either have the wrong distribution (i.e. not every value appears with the same probability), or the output could be too regular. For example, the sequence $X_n = n \bmod m$ hits every value equally often in the long run, but it shows none of the fluctuations which are typical for a sequence of real random numbers. For this reason, statistical tests should be performed as two-sided tests when the distribution of the output of a PRNG is being tested.

Example 1.6 Assume that we have a PRNG with $m = 1024$ possible output values and that we perform a chi-squared test for the hypothesis

$$P(X_i \in \{64j, 64j+1, \dots, 64j+63\}) = 1/16$$

for $j = 0, 1, \dots, 15$.

If we consider a sample $X_1, X_2, \dots, X_N$, the test statistic of the chi-squared test is computed from the observed numbers of samples in each block, given by

$$O_j = \#\{i \mid 64j \leq X_i < 64(j+1)\}.$$

The expected count for block j , assuming that (1.1) holds, is

$$E_j = N \cdot 64/1024 = N/16$$

for $j = 0, 1, \dots, 15$ and the test statistic of the corresponding chi-squared test is

$$Q = \sum_{j=0}^{15} \frac{(O_j - E_j)^2}{E_j}.$$

For large sample size N , and under the hypothesis (1.1), the value Q follows a χ^2 -distribution with 15 degrees of freedom. Some quantiles of this distribution are:

q	6.262	7.261	...	24.996	27.488
$P(Q \leq q)$	0.025	0.05	...	0.95	0.975

Thus, for a one-sided test with significance level $1 - \alpha = 95\%$ we would reject the hypothesis if $Q > 24.996$. In contrast, for a two-sided test with significance level $1 - \alpha = 95\%$, we would reject the hypothesis if either $Q < 6.262$ or $Q > 27.488$.

We consider two different test cases: first, if $X_n = n \bmod 1024$ for $n = 1, 2, \dots, N = 10^6$, we find $Q = 0.244368$. Since the series is very regular, the value of Q is very low. The one-sided test would accept this sequence as being uniformly distributed, whereas the two-sided test would reject the sequence.

Secondly, we consider $X_n = n \bmod 1020$ for $n = 1, 2, \dots, N = 10^6$. Since this series never takes the values 1021 to 1023, the distribution is wrong and we expect a large value of Q . Indeed, for this case we get $Q = 232.5864$ and thus both versions of the test reject this sequence.

Random number generators used in practice, and even the LCG for large enough values of m , pass statistical tests for the distribution of the output samples without problems.

1.1.2.3 Independence of samples

Another aspect of the quality of PRNGs is the possibility of statistical dependence between consecutive samples. For example, in the LCG each output sample is a deterministic function of the previous sample and thus consecutive samples are clearly dependent. To some extent this problem is shared by all PRNGs.

An easy way to visualise the dependence between pairs of consecutive samples is a scatter plot of the points (X_i, X_{i+1}) for $i = 1, 2, \dots, N - 1$. A selection of such plots is shown in Figure 1.1. Figure 1.1(a) illustrates what kind of plot one would expect if $X_i \sim \mathcal{U}[0, 1]$ was a true i.i.d. sequence. The remaining panels correspond to different variants of the LCG. Figure 1.1(b) (using $m = 81$) clearly illustrates that each X_i can only be followed by exactly one value X_{i+1} . While the same is true for Figure 1.1(c) and (d) (using $m = 1024$ and $m = 2^{32}$, respectively), the dependence is much convoluted there and in particular the structure of Figure 1.1(d) is visually indistinguishable from the structure of Figure 1.1(a).

One method for constructing PRNGs where X_{i+1} is not a function of X_i is to use a function $f(X_i)$ of the state, instead of the state X_i itself, as the output of the PRNG. Here, $f: \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, \tilde{m} - 1\}$ is a map where $\tilde{m} < m$ and where the same number of pre-images is mapped to each output value. Then a uniform distribution of X_i will be mapped to a uniform distribution for $f(X_i)$ but the output $f(X_{i+1})$ is not a function of the previous output $f(X_i)$. This allows to construct random number generators with some degree of independence between consecutive values.

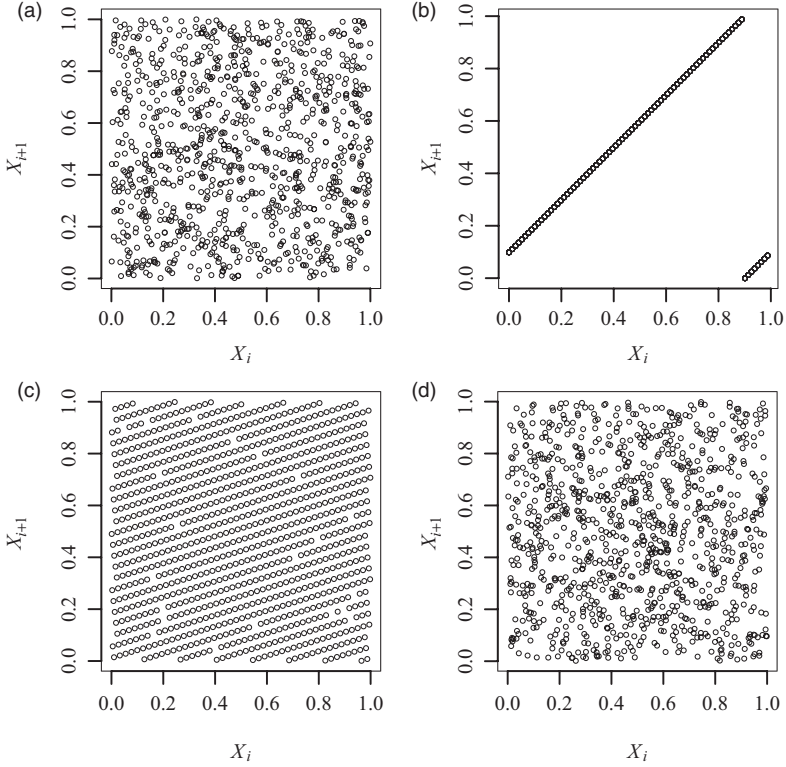


Figure 1.1 Scatter plots to illustrate the correlation between consecutive outputs X_i and X_{i+1} of different pseudo random number generators. The random number generators used are the `runif` function in R (a), the LCG with $m = 81$, $a = 1$ and $c = 8$ (b), the LCG with $m = 1024$, $a = 401$, $c = 101$ (c) and finally the LCG with parameters $m = 2^{32}$, $a = 1\,664\,525$, $c = 1\,013\,904\,223$ (d). Clearly the output in the second and third example does not behave like a sequence of independent random variables.

One way to quantify the independence of the output samples of a PRNG is the following criterion.

Definition 1.7 A periodic sequence $(X_n)_{n \in \mathbb{N}}$ with values in a finite set S and period length P is k -dimensionally equidistributed, if every possible subsequence $x = (x_1, \dots, x_k) \in S^k$ of length k occurs equally often in the sequence X , that is if

$$N_x = \#\{i \mid 0 \leq i < P, X_{i+1} = x_1, \dots, X_{i+k} = x_k\}$$

does not depend on x .

A random number generator is good, if the output is k -dimensionally equidistributed for large values of k .

1.1.3 Pseudo random number generators in practice

This section contains advice on using PRNGs in practice.

First, it is normally a bad idea to implement your own PRNG: finding a good algorithm for pseudo random number generation is a difficult problem, and even when an algorithm is available, given the nature of the generated output, it can be a challenge to spot and remove all mistakes in the implementation. Therefore, it is advisable to use a well-established method for random number generation, typically the random number generator built into a well-known software package or provided by a well-established library.

A second consideration concerns the rôle of the seed. While different PRNGs differ greatly in implementation details, they all use a seed (like the value X_0 in algorithm 1.2) to initialise the state of the random number generator. Often, when non-predictability is required, it is useful to set the seed to some volatile quantity (like the current time) to get a different sequence of random numbers for different runs of the program. At other times it can be more useful to get reproducible results, for example to aid debugging or to ensure repeatability of published results. In these cases, the seed should be set to a known, fixed value.

Finally, PRNGs like the LCG described above often generate a sequence which behaves like a sequence of independent random numbers, uniformly distributed on a finite set $\{0, 1, \dots, m-1\}$ for a big value of m . In contrast, most applications require a sequence of independent, $\mathcal{U}[0, 1]$ -distributed random variables, that is a sequence of i.i.d. values which are uniformly distributed on the real interval $[0, 1]$. We can obtain a sequence $(U_n)_{n \in \mathbb{N}}$ of pseudo random numbers to replace an i.i.d. sequence of $\mathcal{U}[0, 1]$ random variables by setting

$$U_n = \frac{X_n + 1}{m + 1},$$

where $(X_n)_{n \in \mathbb{N}}$ is the output of the PRNG. The output U_n can only take the m different values

$$\frac{1}{m+1}, \frac{2}{m+1}, \dots, \frac{m}{m+1}$$

and thus U_n is not exactly uniformly distributed on the continuous interval $[0, 1]$. But, since the possible values are evenly spaced inside the interval $[0, 1]$ and since each of these values has the same probability, the distribution of U_n is a reasonable approximation to a uniform distribution on $[0, 1]$. This is particularly true since computers can only represent finitely many real numbers exactly.

This concludes our discussion of how a replacement for an i.i.d. sequence of $\mathcal{U}[0, 1]$ -distributed random numbers can be generated on a computer.

1.2 Discrete distributions

Building on the methods from Section 1.1, in this and the following sections we will study methods to transform an i.i.d. sequence of $\mathcal{U}[0, 1]$ -distributed random variables

into an i.i.d. sequence from a prescribed target distribution. The methods from the previous section were inexact, since the output of a PRNG is not ‘truly random’. In contrast, the transformations described in this and the following sections can be carried out with complete mathematical rigour. We will discuss different methods for generating samples from a given distribution, applicable to different classes of target distributions. In this section we concentrate on the simplest case where the target distribution only takes finitely or countably infinitely many values.

As a first example, we consider the uniform distribution on the set $A = \{0, 1, \dots, n-1\}$, denoted by $\mathcal{U}\{0, 1, \dots, n-1\}$. Since the set A has n elements, a random variable X with $X \sim \mathcal{U}\{0, 1, \dots, n-1\}$ satisfies

$$P(X = k) = \frac{1}{n}$$

for all $k \in A$. To generate samples from such a random variable X , at first it may seem like a good idea to just use a PRNG with state space A , for example the LCG with modulus $m = n$. But considering the fact that the maximal period length of a PRNG is restricted to the size of the state space, it becomes clear that this is not a good idea. Instead we will follow the approach to first generate a continuous sample $U \sim \mathcal{U}[0, 1]$ and then to transform this sample into the required discrete uniform distribution. A method to implement this idea is described in the following lemma.

Lemma 1.8 Let $U \sim \mathcal{U}[0, 1]$ and $n \in \mathbb{N}$. Define a random variable X by $X = \lfloor nU \rfloor$, where $\lfloor \cdot \rfloor$ denotes rounding down. Then $X \sim \mathcal{U}\{0, 1, \dots, n-1\}$.

Proof By the definition of X we have

$$P(X = k) = P(\lfloor nU \rfloor = k) = P(nU \in [k, k+1)) = P\left(U \in \left[\frac{k}{n}, \frac{k+1}{n}\right)\right)$$

for all $k = 0, 1, \dots, n-1$.

The uniform distribution $\mathcal{U}[0, 1]$ is characterised by the fact that $U \sim \mathcal{U}[0, 1]$ satisfies

$$P(U \in [a, b]) = b - a$$

for all $0 \leq a \leq b \leq 1$. Also, since U is a continuous distribution, we have $P(U = x) = 0$ for all $x \in [0, 1]$ and thus the boundary points of the interval $[a, b]$ can be included or excluded without changing the probability. Using these results, we find

$$P(X = k) = P\left(U \in \left[\frac{k}{n}, \frac{k+1}{n}\right)\right) = \frac{k+1}{n} - \frac{k}{n} = \frac{1}{n}$$

for all $k = 0, 1, \dots, n-1$. This completes the proof. $\square$

Another common problem related to discrete distributions is the problem of constructing random events which occur with a given probability p . Such events will,

for example, be needed in the rejection algorithms considered in Section 1.4. There are many fascinating aspects to this problem, but here we will restrict ourselves to the simplest case where the probability p is known explicitly and where we have access to $\mathcal{U}[0, 1]$ -distributed random variables. This case is considered in the following lemma.

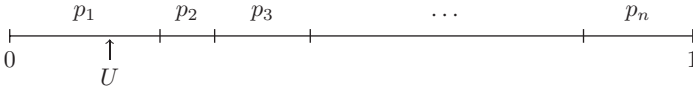
Lemma 1.9 Let $p \in [0, 1]$ and $U \sim \mathcal{U}[0, 1]$ and define the event E as $E = \{U \leq p\}$. Then $P(E) = p$.

Proof We have

$$P(E) = P(U \leq p) = P(U \in [0, p]) = p - 0 = p.$$

This completes the proof. $\square$

The idea underlying lemmas 1.8 and 1.9 can be generalised to sample from arbitrary distributions on a finite set A . Let $A = \{a_1, \dots, a_n\}$ where $a_i \neq a_j$ for $i \neq j$ and let $p_1, \dots, p_n \geq 0$ be given with $\sum_{i=1}^n p_i = 1$. Assume that we want to generate random values $X \in A$ with $P(X = a_i) = p_i$ for $i = 1, 2, \dots, n$. Since the p_i sum up to 1, we can split the unit interval $[0, 1]$ into disjoint sub-intervals lengths $p_1, \dots, p_n$.



With this arrangement, if we choose $U \in [0, 1]$ uniformly, the value of U lies in the i th subinterval with probability p_i . Thus, we can choose X to be the a_i corresponding to the subinterval which contains U . This idea is formalised in the following lemma.

Lemma 1.10 Assume $A = \{a_i \mid i \in I\}$ where either $I = \{1, 2, \dots, n\}$ for some $n \in \mathbb{N}$ or $I = \mathbb{N}$, and where $a_i \neq a_j$ whenever $i \neq j$. Let $p_i \geq 0$ be given for $i \in I$ with $\sum_{i \in I} p_i = 1$. Finally let $U \sim \mathcal{U}[0, 1]$ and define

$$K = \min \left\{ k \in I \mid \sum_{i=1}^k p_i \geq U \right\}. \quad (1.2)$$

Then $X = a_K \in A$ satisfies $P(X = a_k) = p_k$ for all $k \in I$.

Proof We have

$$\begin{aligned} P(X = a_k) &= P(K = k) = P\left(\sum_{i=1}^{k-1} p_i < U, \sum_{i=1}^k p_i \geq U\right) \\ &= P\left(U \in \left(\sum_{i=1}^{k-1} p_i, \sum_{i=1}^k p_i\right]\right) = \sum_{i=1}^k p_i - \sum_{i=1}^{k-1} p_i = p_k \end{aligned}$$

for all $k \in I$, where we interpret the sum $\sum_{i=1}^0 p_i$ for $k = 1$ as 0. This completes the proof. $\square$

The numerical method described by lemma 1.10 requires that we find the index K of the subinterval which contains U . The most efficient way to do this is to find a function φ which maps the boundaries of the subintervals to consecutive integers and then to consider the rounded value $\lfloor \varphi(I) \rfloor$. This approach is taken in lemma 1.8 and also in the following example.

Example 1.11 The geometric distribution, describing the number X of individual trials with probability p until the first success, has probability weights $P(X = i) = p^{i-1}(1 - p) = p_i$ for $i \in \mathbb{N}$. We can use lemma 1.10 with $a_i = i$ for all $i \in \mathbb{N}$ to generate samples from this distribution.

For the weights p_i , the value sum in equation (1.2) can be determined explicitly: using the formula for geometric sums we find

$$\sum_{i=1}^k p_i = (1 - p) \sum_{i=1}^k p^{i-1} = (1 - p) \frac{1 - p^k}{1 - p} = 1 - p^k.$$

Thus, we can rewrite the event $\sum_{i=1}^k p_i \geq U$ as follows:

$$\begin{aligned} \left\{ U \leq \sum_{i=1}^k p_i \right\} &= \{ U \leq 1 - p^k \} \\ &= \{ p^k \leq 1 - U \} \\ &= \{ k \log(p) \leq \log(1 - U) \} \\ &= \left\{ k \geq \frac{\log(1 - U)}{\log(p)} \right\}. \end{aligned}$$

In the last expression, we had to change the $\leq$ sign into a $\geq$ sign, since we divided by the negative number $\log(p)$. By definition, the K from equation (1.2) is the smallest integer such that $\sum_{i=1}^k p_i \geq U$ is satisfied and thus the smallest integer greater than or equal to $\log(1 - U)/\log(p)$. Thus, the value

$$X = a_K = K = \left\lceil \frac{\log(1 - U)}{\log(p)} \right\rceil,$$

where $\lceil \cdot \rceil$ denotes the operation of rounding up a number to the nearest integer, is geometrically distributed with parameter p .

1.3 The inverse transform method

The inverse transform method is a method which can be applied when the target distribution is one-dimensional, that is to generate samples from a prescribed target

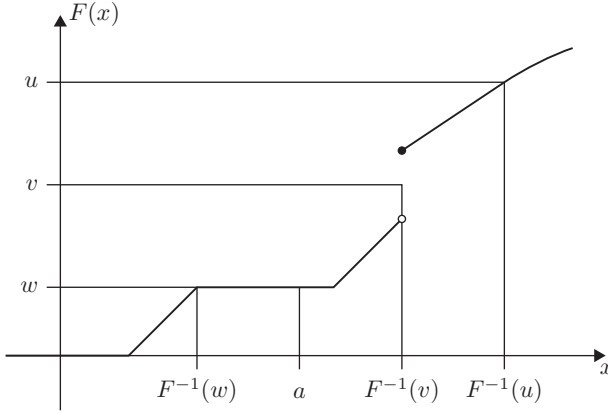


Figure 1.2 Illustration of the inverse F^{-1} of a CDF F . At level u the function F is continuous and injective; here F^{-1} coincides with the usual inverse of a function. The value v falls in the middle of a jump of F and thus has no preimage; $F^{-1}(v)$ is the preimage of the right-hand limit of F and $F(F^{-1}(v)) \neq v$. At level w the function F is not injective, several points map to w ; the preimage $F^{-1}(w)$ is the left-most of these points and we have, for example, $F^{-1}(F(a)) \neq a$.

distribution on the real numbers $\mathbb{R}$. The method uses the cumulative distribution function (CDF) (see Section A.1) to specify the target distribution and can be applied for distributions which have no density.

Definition 1.12 Let F be a distribution function. Then the *inverse* of F is defined by

$$F^{-1}(u) = \inf \{x \in \mathbb{R} \mid F(x) \geq u\}$$

for all $u \in (0, 1)$.

The definition of the inverse of a distribution function is illustrated in Figure 1.2. In the case where F is bijective, that is when F is strictly monotonically increasing and has no jumps, F^{-1} is just the usual inverse of a function. In this case we can find $F^{-1}(u)$ by solving the equation $F(x) = u$ for x . The following algorithm can be used to generate samples from a given distribution, whenever the inverse F^{-1} of the distribution function can be determined.

Algorithm 1.13 (inverse transform method)

input:

the inverse F^{-1} of a CDF F

randomness used:

$U \sim \mathcal{U}[0, 1]$

output:

$$X \sim F$$

1: generate $U \sim \mathcal{U}[0, 1]$

2: return $X = F^{-1}(U)$

This algorithm is very simple and it directly transforms $\mathcal{U}[0, 1]$ -distributed samples into samples with distribution function F . The following proposition shows that the samples X generated by algorithm 1.13 have the correct distribution.

Proposition 1.14 Let $F: \mathbb{R} \rightarrow [0, 1]$ be a distribution function and $U \sim \mathcal{U}[0, 1]$. Define $X = F^{-1}(U)$. Then X has distribution function F .

Proof Using the definitions of X and F^{-1} we find

$$P(X \leq a) = P(F^{-1}(U) \leq a) = P(\inf\{x \mid F(x) \geq U\} \leq a).$$

Since $\inf\{x \mid F(x) \geq U\} \leq a$ holds if and only if $F(a) \geq U$, we can conclude

$$P(X \leq a) = P(F(a) \geq U) = F(a)$$

where the final equality comes from the definition of the uniform distribution on the interval $[0, 1]$. $\square$

Example 1.15 The exponential distribution $\text{Exp}(\lambda)$ has density

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \geq 0 \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

Using integration, we find the corresponding CDF as

$$F(a) = \int_{-\infty}^a f(x) dx = \int_0^a \lambda e^{-\lambda x} dx = -e^{-\lambda x} \Big|_{x=0}^a = 1 - e^{-\lambda a}$$

for all $a \geq 0$. Since this function is strictly monotonically increasing and continuous, F^{-1} is the usual inverse of F . We have

$$1 - e^{-\lambda x} = u \iff -\lambda x = \log(1 - u) \iff x = -\frac{\log(1 - u)}{\lambda}$$

and thus $F^{-1}(u) = -\log(1 - u)/\lambda$ for all $u \in (0, 1)$. Now assume $U \sim \mathcal{U}[0, 1]$. Then proposition 1.14 gives that $X = -\log(1 - U)/\lambda$ is $\text{Exp}(\lambda)$ -distributed. Thus we have found a method to transform $\mathcal{U}[0, 1]$ random variables into $\text{Exp}(\lambda)$ -distributed random variables. The method can be further simplified by using the observation that U and $1 - U$ have the same distribution: if $U \sim \mathcal{U}[0, 1]$, then $-\log(U)/\lambda \sim \text{Exp}(\lambda)$.

Example 1.16 The *Rayleigh distribution* with parameter $\sigma > 0$ has density

$$f(x) = \begin{cases} \frac{x}{\sigma^2} e^{-x^2/2\sigma^2} & \text{if } x \geq 0 \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

For this distribution we find

$$F(a) = \int_0^a \frac{x}{\sigma^2} e^{-x^2/2\sigma^2} dx = -e^{-x^2/2\sigma^2} \Big|_{x=0}^a = 1 - e^{-a^2/2\sigma^2}$$

for all $a \geq 0$. Solving the equation $u = F(x) = 1 - e^{-x^2/2\sigma^2}$ for x we find the inverse $F^{-1}(u) = x = \sqrt{-2\sigma^2 \log(1-u)}$. By proposition 1.14 we know that $X = \sqrt{-2\sigma^2 \log(1-U)}$ has density f if we choose $U \sim \mathcal{U}[0, 1]$. As in the previous example, we can also write U instead of $1 - U$.

Example 1.17 Let X have density

$$f(x) = \begin{cases} 3x^2 & \text{for } x \in [0, 1] \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

Then

$$F(a) = \int_{-\infty}^a f(x) dx = \begin{cases} 0 & \text{if } a < 0 \\ a^3 & \text{if } 0 \leq a < 1 \text{ and} \\ 1 & \text{for } 1 \leq a. \end{cases}$$

Since F maps $(0, 1)$ into $(0, 1)$ bijectively, F^{-1} is given by the usual inverse function and consequently $F^{-1}(u) = u^{1/3}$ for all $u \in (0, 1)$. Thus, by proposition 1.14, if $U \sim \mathcal{U}[0, 1]$, the cubic root $U^{1/3}$ has the same distribution as X .

Example 1.18 Let X be discrete with $P(X = 0) = 0.6$ and $P(X = 1) = 0.4$. Then

$$F(a) = \begin{cases} 0 & \text{if } a < 0 \\ 0.6 & \text{if } 0 \leq a < 1 \text{ and} \\ 1 & \text{if } 1 \leq a. \end{cases}$$

Using the definition of F^{-1} we find

$$F^{-1}(u) = \begin{cases} 0 & \text{if } 0 < u \leq 0.6 \text{ and} \\ 1 & \text{if } 0.6 < u < 1. \end{cases}$$

By proposition 1.14 we can construct a random variable X with the correct distribution from $U \sim \mathcal{U}[0, 1]$, by setting

$$X = \begin{cases} 0 & \text{if } U \leq 0.6 \text{ and} \\ 1 & \text{if } U > 0.6. \end{cases}$$

The inverse transform method can always be applied when the inverse F^{-1} is easy to evaluate. For some distributions like the normal distribution this is not the case, and the inverse transform method cannot be applied directly. The method can be applied (but may not be very useful) for discrete distributions such as in example 1.18. The main restriction of the inverse transform method is that distribution functions only exist in the one-dimensional case. For distributions on $\mathbb{R}^d$ where $d > 1$, more sophisticated methods are required.

1.4 Rejection sampling

The rejection sampling method is a more advanced, and very popular, method for random number generation. Several aspects make this method different from basic methods such as inverse transform method discussed in the previous section. First, rejection sampling is not restricted to $\mathcal{U}[0, 1]$ -distributed input samples. The method is often used in multi-stage approaches where different methods are used to generate samples of approximately the correct distribution and then rejection sampling is used to convert these samples to follow the target distribution exactly. Secondly, while we state the method here only for distributions on the Euclidean space $\mathbb{R}^d$, the rejection sampling method can be generalised to work on very general spaces. Finally, a random and potentially large number of input samples is required to generate one output sample in the rejection method. As a consequence, the efficiency of the method becomes a concern.

1.4.1 Basic rejection sampling

In this section we introduce the fundamental idea that all rejection algorithms are based on. We start by presenting the basic algorithm which forms the prototype of the methods presented later.

Algorithm 1.19 (basic rejection sampling)

input:

- a probability density g (the *proposal density*),
- a function p with values in $[0, 1]$ (the *acceptance probability*)

randomness used:

- X_n i.i.d. with density g (the *proposals*),
- $U_n \sim \mathcal{U}[0, 1]$ i.i.d.

output:

a sequence of i.i.d. random variables with density

$$f(x) = \frac{1}{Z} p(x)g(x) \quad \text{where} \quad Z = \int p(x)g(x) dx. \quad (1.3)$$

```

1: for  $n = 1, 2, 3, \dots$  do
2:   generate  $X_n$  with density  $g$ 
3:   generate  $U_n \sim \mathcal{U}[0, 1]$ 
4:   if  $U_n \leq p(X_n)$  then
5:     output  $X_n$ 
6:   end if
7: end for

```

The effect of the random variables U_n in the algorithm is to randomly decide whether to output or to ignore the value X_n : the value X_n is output with probability $p(X_n)$, and using the trick from lemma 1.9 we use the event $\{U \leq p(X_n)\}$ to decide whether or not to output the value. In the context of rejection sampling, the random variables X_n are called *proposals*. If the proposal X_n is chosen for output, that is if $U_n \leq p(X_n)$, we say that X_n is *accepted*, otherwise we say that X_n is *rejected*.

Proposition 1.20 For $k \in \mathbb{N}$, let X_{N_k} denote the k th output of algorithm 1.19. Then the following statements hold:

- (a) The elements of the sequence $(X_{N_k})_{k \in \mathbb{N}}$ are i.i.d. with density f given by (1.3).
- (b) Each proposal is accepted with probability Z ; the number of proposals required to generate each X_{N_k} is geometrically distributed with mean $1/Z$.

Proof For fixed n , the probability of accepting X_n is

$$P(U_n \leq p(X_n)) = \int p(x)g(x) dx = Z, \quad (1.4)$$

where Z is the constant defined in equation (1.3). Since the decisions whether to accept X_n for different n are independent, the time until the first success is geometrically distributed with mean $1/Z$ as required. This completes the proof of the second statement.

For the proof of the first statement, first note that the indices $N_1, N_2, N_3, \dots$ of the accepted X_n are random. If we let $N_0 = 0$, we can write

$$N_k = \min \{n \in \mathbb{N} \mid n > N_{k-1}, U_n \leq p(X_n)\}$$

for all $k \in \mathbb{N}$. If we consider the distribution of X_{N_k} conditional on the value of N_{k-1} , we find

$$\begin{aligned}
 & P(X_{N_k} \in A \mid N_{k-1} = n) \\
 &= \sum_{m=1}^{\infty} P(N_k = n + m, X_{n+m} \in A \mid N_{k-1} = n) \\
 &= \sum_{m=1}^{\infty} P(U_{n+1} > p(X_{n+1}), \dots, U_{n+m-1} > p(X_{n+m-1}), \\
 &\quad U_{n+m} \leq p(X_{n+m}), X_{n+m} \in A \mid N_{k-1} = n) \\
 &= \sum_{m=1}^{\infty} P(U_{n+1} > p(X_{n+1})) \cdots P(U_{n+m-1} > p(X_{n+m-1})) \cdot \\
 &\quad P(U_{n+m} \leq p(X_{n+m}), X_{n+m} \in A).
 \end{aligned}$$

Here we used the fact that all the probabilities considered in the last expression are independent of the value of N_{k-1} . Similar to (1.4) we find

$$P(U_n \leq p(X_n), X_n \in A) = \int_A p(x)g(x)dx$$

and consequently we have

$$\begin{aligned}
 & P(X_{N_k} \in A \mid N_{k-1} = n) \\
 &= \sum_{m=1}^{\infty} (1 - Z)^{m-1} \int_A p(x)g(x)dx \\
 &= \frac{1}{Z} \int_A p(x)g(x)dx
 \end{aligned}$$

by the geometric series formula. Since the right-hand side does not depend on n , we can conclude

$$P(X_{N_k} \in A) = \frac{1}{Z} \int_A p(x)g(x)dx$$

and thus we find that X_{N_k} has density pg/Z .

To see that the X_{N_k} are independent we need to show that

$$P(X_{N_1} \in A_1, \dots, X_{N_k} \in A_k) = \prod_{i=1}^k P(X_{N_i} \in A_i)$$

for all sets $A_1, \dots, A_k$ and for all $k \in \mathbb{N}$. This can be done by summing up the probabilities for the cases $N_1 = n_1, \dots, N_k = n_k$, similar to the first part of the proof, but we omit this tedious calculation here. $\square$

Example 1.21 Let $X \sim \mathcal{U}[-1, +1]$ and accept X with probability

$$p(X) = \sqrt{1 - X^2}.$$

Then, by proposition 1.20, the accepted samples have density

$$f(x) = \frac{1}{Z} p(x) g(x) = \frac{1}{Z} \cdot \sqrt{1 - x^2} \cdot \frac{1}{2} \mathbb{1}_{[-1, +1]}(x),$$

where we use the indicator function notation from equation (A.7) to get the abbreviation

$$\mathbb{1}_{[-1, +1]}(x) = \begin{cases} 1 & \text{if } x \in [-1, +1] \text{ and} \\ 0 & \text{otherwise} \end{cases}$$

and

$$Z = \int_{\mathbb{R}} \sqrt{1 - x^2} \cdot \frac{1}{2} \mathbb{1}_{[-1, +1]}(x) dx = \frac{1}{2} \int_{-1}^1 \sqrt{1 - x^2} dx = \frac{1}{2} \cdot \frac{\pi}{2} = \frac{\pi}{4}.$$

Combining these results, we find that the density f of accepted samples is given by

$$f(x) = \frac{2}{\pi} \sqrt{1 - x^2} \mathbb{1}_{[-1, +1]}(x).$$

The graph of the density f forms a semicircle and the resulting distribution is known as Wigner's semicircle distribution.

One important property of the rejection algorithm 1.19 is that none of the steps in the algorithm makes any reference to the normalisation constant Z . Thus, we do not need to compute the value of Z in order to apply this algorithm. We will see that this fact, while looking like a small detail at first glance, is extremely useful in practical applications.

1.4.2 Envelope rejection sampling

The basic rejection sampling algorithm 1.19 from the previous section is usually applied by choosing the acceptance probabilities p so that the density f of the output values, given by (1.3), coincides with a given target distribution. The resulting algorithm can be written as in the following.

Algorithm 1.22 (envelope rejection sampling)

input:

- a function f with values in $[0, \infty)$ (the non-normalised *target density*),
- a probability density g (the *proposal density*),
- a constant $c > 0$ such that $f(x) \leq c g(x)$ for all x

randomness used:

- X_n i.i.d. with density g (the *proposals*),
- $U_n \sim \mathcal{U}[0, 1]$ i.i.d.

output:

- a sequence of i.i.d. random variables with density

$$\tilde{f}(x) = \frac{1}{Z_f} f(x) \quad \text{where} \quad Z_f = \int f(x) dx$$

```

1: for  $n = 1, 2, 3, \dots$  do
2:   generate  $X_n$  with density  $g$ 
3:   generate  $U_n \sim \mathcal{U}[0, 1]$ 
4:   if  $c g(X_n) U_n \leq f(X_n)$  then
5:     output  $X_n$ 
6:   end if
7: end for

```

The assumption in the algorithm is that we can already sample from the distribution with probability density g , but we would like to generate samples from the distribution with density $\tilde{f}$ instead. Normally, f will be chosen to be a probability density and in this case we have $\tilde{f} = f$, but in some situations the normalising constant Z_f is difficult to obtain and due to the distinction between f and $\tilde{f}$, in these situations the algorithm can still be applied. The rejection mechanism employed in algorithm 1.22 is illustrated in Figure 1.3. The function $c g$ is sometimes called an ‘envelope’ for f .

Proposition 1.23 Let X_{N_k} for $k \in \mathbb{N}$ denote the k th output value of algorithm 1.22 with (non-normalised) target density f . Then the following statements hold:

- (a) The elements of the sequence $(X_{N_k})_{k \in \mathbb{N}}$ are i.i.d. with density $\tilde{f}$.
- (b) Each proposal is accepted with probability Z_f/c ; the number $M_k = N_k - N_{k-1}$ of proposals required to generate each X_{N_k} is geometrically distributed with mean $E(M_k) = c/Z_f$.

Proof Algorithm 1.22 coincides with algorithm 1.19 where the acceptance probability p is chosen as

$$p(x) = \begin{cases} \frac{f(x)}{c g(x)} & \text{if } g(x) > 0 \text{ and} \\ 1 & \text{otherwise.} \end{cases}$$

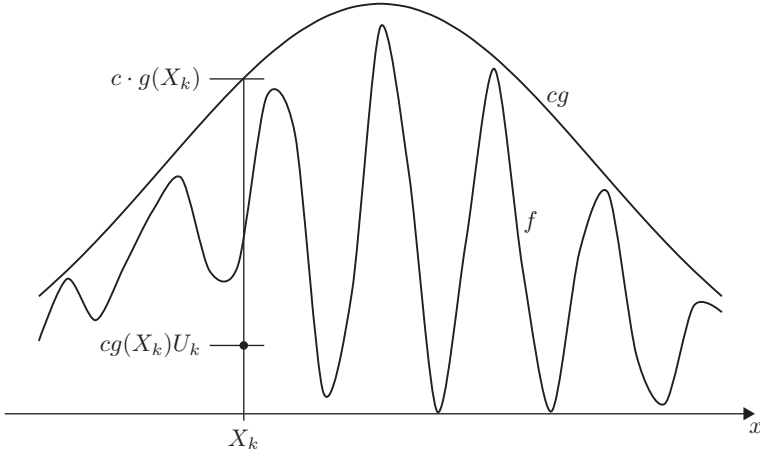


Figure 1.3 Illustration of the envelope rejection sampling method from algorithm 1.22. The proposal $(X_k, cg(X_k) U_k)$ is accepted, if it falls into the area underneath the graph of f . In Section 1.4.4 we will see that the proposal is distributed uniformly on the area under the graph of cg .

In this situation, the normalisation constant Z from (1.3) is given by:

$$Z = \int p(x)g(x) dx = \int \frac{f(x)}{cg(x)} g(x) dx = \frac{1}{c} \int f(x) dx = Z_f/c.$$

From proposition 1.20 we then know that the output of algorithm 1.19 is an i.i.d. sequence with density

$$\frac{1}{Z} pg = \frac{c}{Z_f} \frac{f}{cg} g = \frac{1}{Z_f} f$$

and that the required number of proposals to generate one output sample is geometrically distributed with mean $1/Z = c/Z_f$. $\square$

Example 1.24 We can use rejection sampling to generate samples from the half-normal distribution with density

$$f(x) = \begin{cases} \frac{2}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) & \text{if } x \geq 0 \text{ and} \\ 0 & \text{otherwise.} \end{cases} \quad (1.5)$$

If we assume that the proposals are $\text{Exp}(\lambda)$ -distributed, then the density of the proposals is

$$g(x) = \begin{cases} \lambda \exp(-\lambda x) & \text{if } x \geq 0 \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

In order to apply algorithm 1.22 we need to determine a constant $c > 0$ such that $f(x) \leq cg(x)$ for all $x \in \mathbb{R}$. For $x < 0$ we have $f(x) = g(x) = 0$. For $x \geq 0$ we have

$$\frac{f(x)}{g(x)} = \frac{2}{\sqrt{2\pi\lambda}} \exp\left(-\frac{x^2}{2} + \lambda x\right).$$

It is easy to check that the quadratic function $-x^2/2 + \lambda x$ attains its maximum at $x = \lambda$. Thus we have

$$\frac{f(x)}{g(x)} \leq c^*$$

for all $x \geq 0$, where

$$c^* = \frac{2}{\sqrt{2\pi\lambda}} \exp\left(-\frac{\lambda^2}{2} + \lambda \cdot \lambda\right) = \sqrt{\frac{2}{\pi\lambda^2}} \exp(\lambda^2/2).$$

Consequently, any $c \geq c^*$ satisfies the condition $f \leq cg$. From proposition 1.23 we know that the average number of proposals required for generating one sample, and thus the computational cost, is proportional to c . Thus we should choose c as small as possible and $c = c^*$ is the optimal choice.

Given our choice of g and c , the acceptance criterion from algorithm 1.22 can be simplified as follows:

$$\begin{aligned} cg(x)U &\leq f(x) \\ \iff \sqrt{\frac{2}{\pi\lambda^2}} \exp\left(\frac{\lambda^2}{2}\right) \lambda \exp(-\lambda x) U &\leq \frac{2}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \\ \iff U &\leq \exp\left(-\frac{x^2}{2} + \lambda x - \frac{\lambda^2}{2}\right) \\ \iff U &\leq \exp\left(-\frac{1}{2}(x - \lambda)^2\right). \end{aligned}$$

This leads to the following algorithm for generating samples from the half-normal distribution:

- 1: **for** $n = 1, 2, 3, \dots$ **do**
- 2: generate $X_n \sim \text{Exp}(\lambda)$
- 3: generate $U_n \sim \mathcal{U}[0, 1]$

```

4:   if  $U_n \leq \exp(-\frac{1}{2}(X_n - \lambda)^2)$  then
5:       output  $X_n$ 
6:   end if
7: end for

```

Finally, since the density f is the density of a standard-normal distribution conditioned on being positive, and since the normal distribution is symmetric, we can generate standard normal distributed values by randomly choosing X_n or $-X_n$, both with probability $1/2$, for each accepted sample.

In algorithm 1.22, we can choose the density g of the proposal distribution in order to maximise efficiency of the method. The only constraint is that we need to be able to find the constant c . This condition implies, for example, that the support of g cannot be smaller than the support of f , that is we need $g(x) > 0$ whenever $f(x) > 0$. The average cost of generating one sample is given by the average number of proposals required times the cost for generating each proposal. Therefore the algorithm is efficient, if the following two conditions are satisfied:

- (a) There is an efficient method to generate the proposals X_i . This affects the choice of the proposal density g .
- (b) The average number c/Z_f of proposals required to generate one sample is small. This number is influenced by the value of c and, since the possible choices of c depend on g , also by the proposal density g .

1.4.3 Conditional distributions

The conditional distribution $P_{X|X \in A}$ corresponds to the remaining randomness in X when we already know that $X \in A$ occurred (see equation (A.4) for details). Sampling from a conditional distribution can be easily done by rejection sampling. The basic result is the following.

Algorithm 1.25 (rejection sampling for conditional distributions)

```

input:
  a set  $A$  with  $P(X \in A) > 0$ 
randomness used:
  a sequence  $X_n$  of i.i.d. copies of  $X$  (the proposals)
output:
  a sequence of i.i.d. random variables with distribution  $P_{X|X \in A}$ 
1: for  $n = 1, 2, 3, \dots$  do
2:   generate  $X_n$ 
3:   if  $X_n \in A$  then
4:     output  $X_n$ 
5:   end if
6: end for

```

Proposition 1.26 Let X be a random variable and let A be a set. Furthermore, let X_{N_k} for $k \in \mathbb{N}$ denote the k th output value of algorithm 1.25. Then the following statements hold:

- (a) The elements of the sequence $(X_{N_k})_{k \in \mathbb{N}}$ are i.i.d. and satisfy

$$P(X_{N_k} \in B) = P(X \in B | X \in A)$$

for all $k \in \mathbb{N}$ and all sets B .

- (b) The number $M_k = N_k - N_{k-1}$ of proposals required to generate each X_{N_k} is geometrically distributed with mean $E(M_k) = 1/P(X \in A)$.

Proof Algorithm 1.25 is a special case of algorithm 1.19 where the acceptance probability is chosen as $p(x) = \mathbb{1}_A(x)$. For this choice of p , the decision whether or not to accept the proposal given the value of X_n is deterministic and thus we can omit generation of the auxiliary random variables U_n in the algorithm.

Now assume that the distribution of X has a density g . Using equation (1.3) we then find

$$Z = \int \mathbb{1}_A(x)g(x) dx = P(X \in A)$$

and by proposition 1.20 we have

$$P(X_{N_k} \in B) = \frac{\int_B \mathbb{1}_A(x)g(x) dx}{Z} = \frac{P(X \in B \cap A)}{P(X \in A)} = P(X \in B | X \in A).$$

A similar proof gives the result in the case where X does not have a density. This completes the proof of the first statement of the proposition. The second statement is a direct consequence of proposition 1.20. $\square$

The method presented in algorithm 1.25 works well if $p = P(X \in A)$ is not too small; the time required for producing a single output sample is proportional to $1/p$.

Example 1.27 We can use algorithm 1.25 to generate samples $X \sim \mathcal{N}(0, 1)$, conditioned on $X \geq a$. We simply have to repeat the following two steps until enough samples are output:

- (a) generate $X \sim \mathcal{N}(0, 1)$;
- (b) if $X \geq a$, output X .

The efficiency of this method depends on the value of a . The following table shows the average number $\mathbb{E}(N_a)$ of samples required to generate one output sample for different values of a , rounded to the nearest integer:

a	1	2	3	4	5	6
$\mathbb{E}(N_a)$	6	44	741	31 574	3 488 556	1 013 594 692

The table shows that the method will be slow even for moderate values of a . For $a \geq 5$ the required number of samples is so large that the method will likely be no longer practical.

For conditions with very small probabilities, rejection sampling can still be used to generate samples from the conditional distribution, but we have to use the full rejection sampling algorithm 1.22 instead of the simplified version from algorithm 1.25. This is illustrated in the following example.

Example 1.28 We can use algorithm 1.22 to generate samples from the conditional distribution of $X \sim \mathcal{N}(0, 1)$, conditioned on $X \geq a > 0$. The density of the conditional distribution is

$$\tilde{f}(x) = \frac{1}{Z} \exp(-x^2/2) \mathbb{1}_{[a, \infty)}(x) = \frac{1}{Z} f(x),$$

where Z is the normalising constant (we have included the pre-factor $1/\sqrt{2\pi}$ into Z to simplify notation).

We can sample from this distribution using proposals of the form $X = \tilde{X} + a$ where $\tilde{X} \sim \text{Exp}(\lambda)$. This proposal distribution has density

$$g(x) = \lambda \exp(-\lambda(x - a)) \mathbb{1}_{[a, \infty)}(x)$$

and we need to find a constant $c > 0$ such that $f(x) \leq cg(x)$ for all $x \geq a$. Also, we can still choose the parameter λ and, in order to maximise efficiency of the method, we should choose a value of λ such that the shape of g is as similar to the shape of f as possible. In order to achieve this, we choose c and λ so that at $x = a$ both the values and the derivatives of f and cg coincide (see Figure 1.4 for illustration). This leads to the conditions $e^{-a^2/2} = f(a) = cg(a) = c\lambda$ and $-ae^{-a^2/2} = f'(a) = cg'(a) = -c\lambda^2$ and solving these two equations for the two unknowns c and λ gives $\lambda = a$ and $c = e^{-a^2/2}/a$.

Figure 1.4 indicates that for this choice of λ and c , condition $f \leq cg$ will be satisfied. Indeed we find

$$\begin{aligned} \frac{f(x)}{cg(x)} &= \frac{\exp(-x^2/2)}{1/a \exp(-a^2/2) \cdot a \exp(-a(x - a))} \\ &= \exp(-x^2/2 + ax - a^2/2) = \exp\left(-\frac{(x - a)^2}{2}\right) \\ &\leq 1 \end{aligned}$$